Strengths and Challenges to Using AI Agents for Research

U.S. Department of Labor, Chief Evaluation Office

Scott Cunningham, Baylor University

Expert Series · October 14, 2026

Who am I?

Scott Cunningham. Ben H. Williams Professor of Economics at Baylor University, Visiting Professor of Public Policy at Harvard Kennedy School and Amazon consultant

Research interests — mostly applied empirical work:

- Corrections
- Suicide
- Causal inference
- Drug policy
- Online dating
- Sex work
- Mental illness
- AI agents

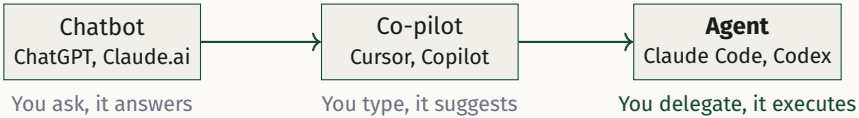
Slides: `scunning.com/files/cunningham_dol_ai_agents_2026-10-14.pdf`

PART 1

What do people most often get wrong about AI agents?

Agents are different because they act

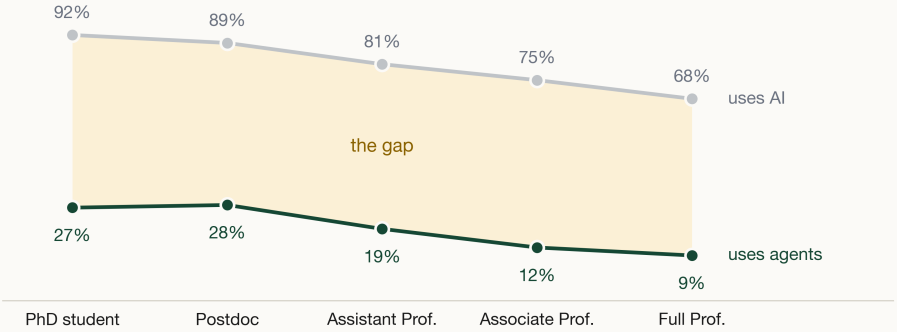
A spectrum of AI use, from passive to autonomous:



An agent reads files, writes code, runs it, reads the output, and decides what to do next across many steps.

AI use is ubiquitous, but use of agents is not

Part of the gap is simply price: chatbots are free, agents are not



Data: Anthropic, May 2026; 1,260 quantitative social scientists. Not a representative sample; read the gradient, not the level.

Agents are not software; they are language models that talk to your computer

- Many people assume an agent like Claude Code, Codex, or Kimi is plugged into the computer, the way a program is.
- It is not: it is the same language model as before, predicting the next token.
- The difference is that its tokens are commands, typed into a terminal the way you or I would type them.

An agent works by talking, so it can only act where typing is accepted.

What the agent actually does

An agent is not just a chatbot with better prose. It acts on your project.

`ls data/`	list files
`rg "minimum wage" .`	search text and code
`grep -r "treated" code/`	search inside files
`curl -O https://site.gov/data.csv`	download from the web
`Rscript estimate.R`	run an R script
`python audit.py`	run a Python script
`stata -b do analysis.do`	run a Stata do-file
`pdflatex paper.tex`	compile a paper or deck
`git diff`	inspect changes
`rm -rf data/`	delete files, permanently
`diskutil eraseDisk ...`	format a hard drive

It reads files, writes files, runs commands, sees errors, revises, and tries again.

A reasoning agent leaves a trace, and the trace is data

What is in a trace

- the reasoning the model wrote to itself before it acted
- every command it ran and every file it opened
- everything it saw come back, including the errors

Why it matters to a researcher

- Claude Code saves each session to a transcript file on your machine.
- A transcript is text, so an agent can read hundreds of them and code what happened.
- This is how I learned what my agents did that their finished output never showed.

The output tells you what the agent concluded. The trace tells you what it did to get there.

A trace from the minimum-wage runs (Kimi, open-weight)

One agent's own account of the choices it made:

> *"Post-2009 period chosen because the federal minimum wage was constant at $7.25, creating clean cross-state variation. Callaway–Sant'Anna drops 15 states already treated in 2010 (no pre-treatment data in the sample). Effective sample: 36 states, 16 treated and 20 never-treated."*

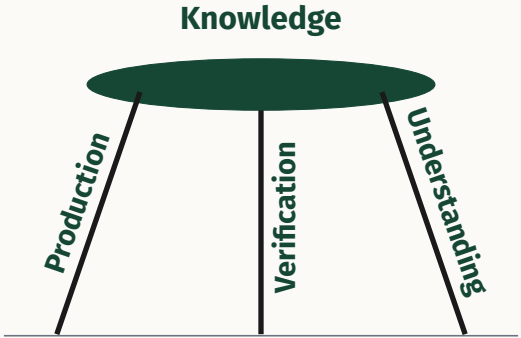
Every choice is in the trace: the window, the estimator, the dropped states. We extract these from every run.

Distillation is why the closed labs now hide the reasoning

- Distillation means training your own model on another model's outputs, including its raw reasoning.
- Competitors, including Chinese labs, were training on the raw reasoning traces of Claude and OpenAI's Codex.
- So both companies stopped publishing raw reasoning, and Claude 4.6 is the last Anthropic model I could see reason.
- You still see the agent's actions, but the reasoning behind them is suppressed.

Every closed model is now taking actions whose reasoning is hidden from you.

Knowledge stands on three legs



Production is one leg of three. Take away verification or understanding, and the chair falls.

PART 2

What advice would you give someone just getting started?

Use agents when the work is valuable, slow, and checkable

My view: delegate to agents when a task satisfies all four conditions.

1. **The output is highly valuable.**
   The stakes justify the setup cost.
2. **The task is time-intensive for humans.**
   Hours or days of skilled effort, not minutes.
3. **The probability humans do it well is low.**
   Difficult to execute at high quality consistently.
4. **The output can be verified.**
   You can inspect, compare, rerun, or source-ground it.

Coding is the canonical example, but this also describes literature review, replication, deck creation, and referee-style review.

Setup links are now an attack surface

Agent workflows often ask you to copy commands into a terminal.

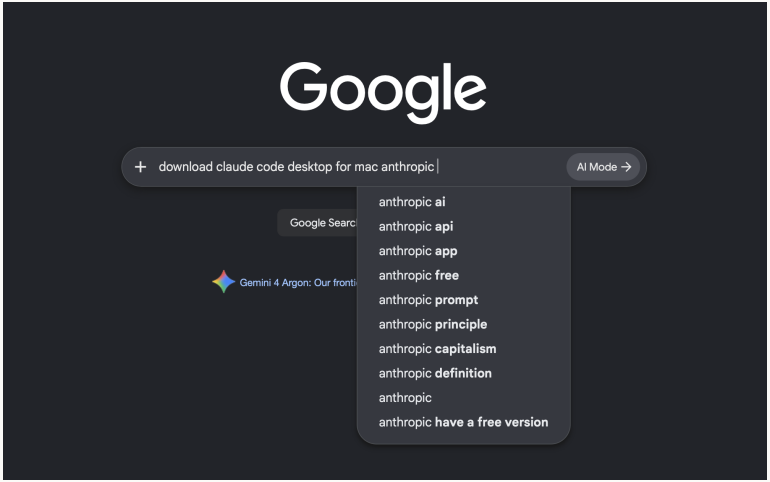
That changes phishing:

- cloned sites can imitate `claude.ai`, GitHub, or package docs
- a copy button can hide a dangerous install command
- agents can browse, compare domains, inspect repos, and explain commands
- do not paste terminal commands you do not understand

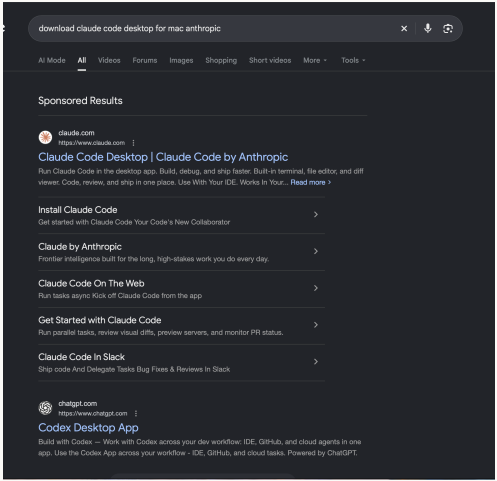
Use agents defensively too: ask them to verify the source before you trust the setup.

Recent threat reports: Google Threat Intelligence on AI-assisted social engineering; Cloud Security Alliance on indirect prompt injection.

Most of us start by searching for it



And the first thing we see is an ad

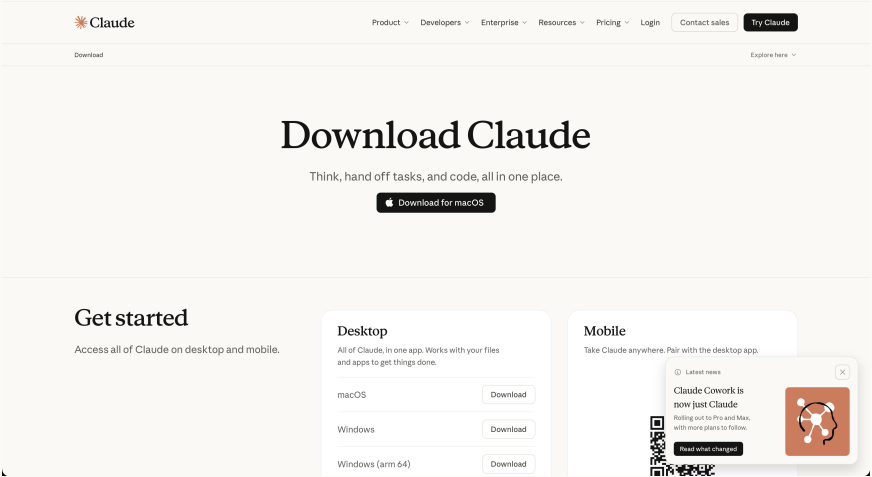


The real download is one of the first things an ad will offer you

- This is a sponsored result: it happens to point to Anthropic, but ads are where fake installers show up.
- Type the address yourself: `claude.com/download`.

Never paste a terminal command you found in an ad or a stranger's post.

Type the address yourself, and you land on the real download page



There are three ways to work with an agent

The desktop app	the easiest start, and it keeps every session
The command line	the agent in a terminal window, closest to the machine
An IDE	the agent next to your files, your code, and a terminal

It is the same agent in all three, and each way of working has strengths and weaknesses.

The desktop app keeps every session, even after you restart your computer

- Every session is listed in the sidebar, and you can reopen any of them.
- Nothing is lost when you shut the laptop or reboot.
- That makes it the easiest place to start.

In the command line, sessions are easier to lose, so save your work to files

- You can re-enter an old session, but in my experience a reboot often loses it.
- Long sessions should be restarted anyway, for a reason on the next slide.
- So have the agent regularly write progress logs to markdown files in the project.

The conversation is temporary. The files on disk are the memory.

A long session fills up with tokens, and the model does not read them evenly

- Every prompt, file, and output in a session is tokens, and the model rereads all of them at every turn.
- Attention does not weigh them equally, so as the session grows, parts get skimmed.
- Even `CLAUDE.md` can fade, and then the agent starts breaking rules you wrote there.

Restart long sessions, and keep what matters on disk rather than in the conversation.

On uneven use of long contexts: Liu et al., “Lost in the Middle” (2024).

An IDE shows you the whole folder, not just the conversation

- An IDE, an integrated development environment, puts your files, an editor, and a terminal in one window.
- What I like most is that I can see the whole folder the agent is working in, not just what it tells me.
- VS Code is free, and it is full of add-ons for Stata, R, Python, LaTeX, and PDFs.

Five steps get VS Code and Claude Code working

1. Download VS Code from `code.visualstudio.com`, typing the address yourself.
2. In VS Code, open Extensions, search for “Claude Code,” and check that the publisher is Anthropic.
3. Install it and restart VS Code.
4. Sign in with your Claude account.
5. Open your whole project folder, never a single file.

Then your first prompt: “Read the files in this folder and tell me what this project is about.”

Adapted from Claes Bäckman, “Practical AI for Academics” (2026).

What `claude –dangerously-skip-permissions` does

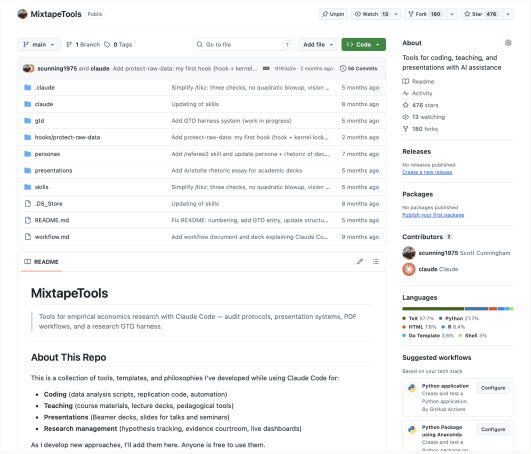
- Typing `claude` in a terminal starts Claude Code right there, in whatever folder the terminal is open to.
- The add-on `–dangerously-skip-permissions` tells it to stop asking before it runs a command or edits a file.
- I use it because it is fast: the agent works in the terminal without stopping for my approval every few seconds.

It also removes the safety check, so start without it, and put guardrails such as hooks in place before you use it.

PART 3

What are the clear-cut uses?

My skills are public, in a repository called MixtapeTools



github.com/scunning1975/MixtapeTools

`/referee2` is one of the skills I use most

- It has Claude Code replicate every analysis file in R, Stata, and Python.
- At each step, it checks that the three versions agree: the same sample, the same coefficients, the same standard errors, and the same F-statistics.
- Any disagreement is a flag I have to resolve before I trust the result.

It is a simple way to verify code I did not write myself.

Why three languages: coding errors are random, not systematic

- The syntax of R, Stata, and Python is very different.
- So a coding error, or a hallucinated one, in one language is unlikely to happen the same way in the others.
- If all three agree, an error would have had to happen three times, in three different ways, with the same result.

Agreement across languages is evidence; agreement within one language is not.

Verification works when errors are imperfectly correlated

No checker is the oracle. The point is to compare independent error processes.

Checker 1	Checker 2	What disagreement reveals
Agent	fresh agent	prompt- or context-specific errors
Agent	human RA	machine vs. human blind spots
R code	Stata/Python code	implementation-specific bugs
Manuscript	source paper	citation and interpretation errors

The standard is zero avoidable errors. One pass is not enough.

Even zero discretion leaves implementation risk

Independent Hallucination Errors. A second experiment.

- Replicate Dias & Fontes (2024, *AEJ: Economic Policy*) on the same CAPS data.
- **96 zero-discretion agents.** Every decision prescribed in advance.
- Vary only what the principal might otherwise vary silently: the package and the covariate set.

Six implementations of the same estimator. Sixteen specifications. Same data, same method, same controls. They should agree.

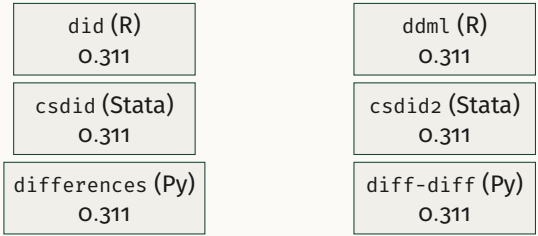
Six packages implement the same estimator differently

Package	Language	Author
`did`	R	Callaway & Sant'Anna
`ddml`	R	Ahrens, Hansen, Schaffer, Wiemann
`csdid`	Stata	Rios-Avila
`csdid2`	Stata	Rios-Avila
`differences`	Python	Dionisi
`diff-diff`	Python	Gerber

All using doubly robust + not-yet-treated controls + universal base period.

Identical specification across all six.

With no covariates, all six packages agree exactly



ATT $\approx$ 0.311
no covariates

The estimator itself is implemented correctly.

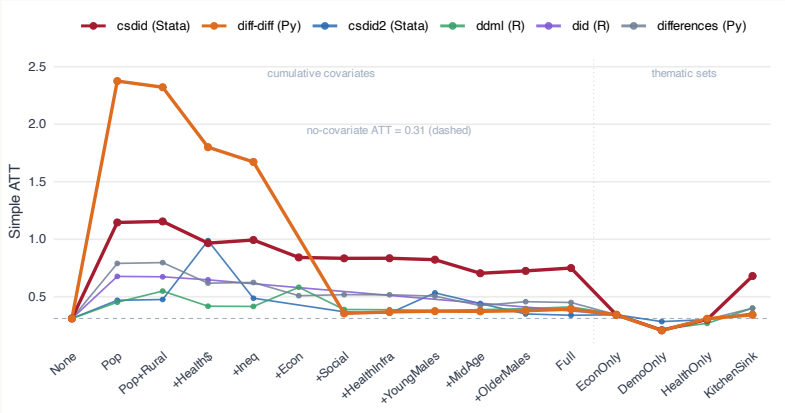
Add one covariate, and the agreement breaks

One covariate: `poptotaltrend` — municipal population × year.

Package	ATT
`ddml` (R)	0.45
`csdid2` (Stata)	0.45
`did` (R)	0.53
`differences` (Python)	0.67
`csdid` (Stata)	1.15
`diff-diff` (Python)	2.38

Same data. Same estimator. Same single covariate. A 5× spread.

The disagreement fans out as covariates accumulate



Each line is one of six packages, plotted across 16 specifications. All start at the same point and then fan out.

The smoking gun: remove that one variable

Run specs using only economics, demographics, or health covariates, excluding `poptotaltrend`.

All six packages agree to within 0.007.

One variable caused the entire 5× spread.

The matrix inversion is the easy part

The propensity-score logit's design matrix X has a column `poptotaltrend` (population $\times$ year). For a 50,000-person municipality in 2010, that single cell is **100,500,000**. The variable lives on the order of 10^{10}.

The other covariates live on the order of 0.01 to 16,000. Mixed in the same X, the condition number of $X'X$ explodes to roughly 3.6×10^{9}.

Below machine epsilon (2.22×10^{-16}): the computer literally cannot distinguish the columns at 64-bit precision. Inverting $X'X$ — which every propensity-score logit needs — produces garbage.

But this is just a logistic regression. If $X'X$ is hard to invert, shouldn't *all six* packages have a hard time with it?

The answer turns out to be no.

Six “logistic regressions” are not the same logistic regression

Package	Lang	Routine called	Optimizer
`did`	R	`glm(probit)`	IRLS
`ddml`	R	`glmnet` Lasso (DR) + `glm(probit)` trim	L1 coord. descent + IRLS
`csdid`	Stata	`probit, iterate(1000) difficult` + `drimp`	Newton–Raphson
`csdid2`	Stata	`probit, difficult ltol(0) tol(1e-5)`	Newton–R., relaxed tol.
`differences`	Py	`statsmodels.Probit().fit()`	statsmodels Newton
`diff-diff`	Py	`statsmodels.Probit()` + `CallawaySantAnna`	statsmodels + internal

Same name on the API. Different optimizer. Different singular-matrix behavior. The user does not see this.

Each row is the actual call inside the package, recovered from the source.

Is this wrong?

What we found:

- `did` *silently* returns all-zero ATTs without raising
- `csdid` requires `iterate(1000) difficult` to nominally converge
- `csdid2` runs with relaxed tolerance: “stabilized but not formally converged”
- `ddml`’s Lasso auto-standardizes inside the learner
- `statsmodels` converges silently — possibly to wrong values

Honest answer: no individual choice is indefensible. Each package author made a reasonable engineering call. But the choice is invisible to the user. The documentation says “logistic regression.”

The new researcher degrees of freedom: sort into language $\rightarrow$ package $\rightarrow$ hidden algorithmic action. Three layers; only the first appears in methods.

The fix: z-score the covariates, or divide `poptotaltrend` by 10^6. The ill-conditioning is the trigger; remove the trigger and most of the divergence collapses.

Hidden actions become detectable only when independent agents replicate one another.

Agents make expensive verification routines cheap enough to run

AI did not magically know the answer.

The workflow made the discrepancy visible:

- many independent agents
- multiple languages
- six packages
- forced comparison to a common target

Agents are useful because they make expensive verification routines cheap enough to run.

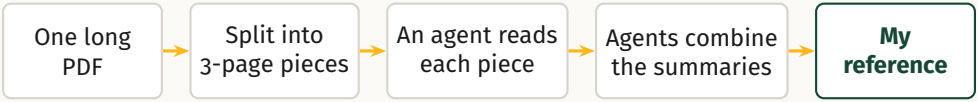
Some files are easy for an LLM to read, and some are hard

Easiest	plain text: `.md`, `.tex`, `.txt`, code files
	small `.csv` files, up to a few thousand rows
	Word, Excel, and Stata files, once converted to text
	short PDFs, though tables come out mangled
Hardest	long or scanned PDFs: convert them to text first

The closer a file is to plain text, the better the agent reads it.

Adapted from Claes Bäckman, “Practical AI for Academics” (2026).

`/split-pdf` turns a long PDF into a reference I can trust



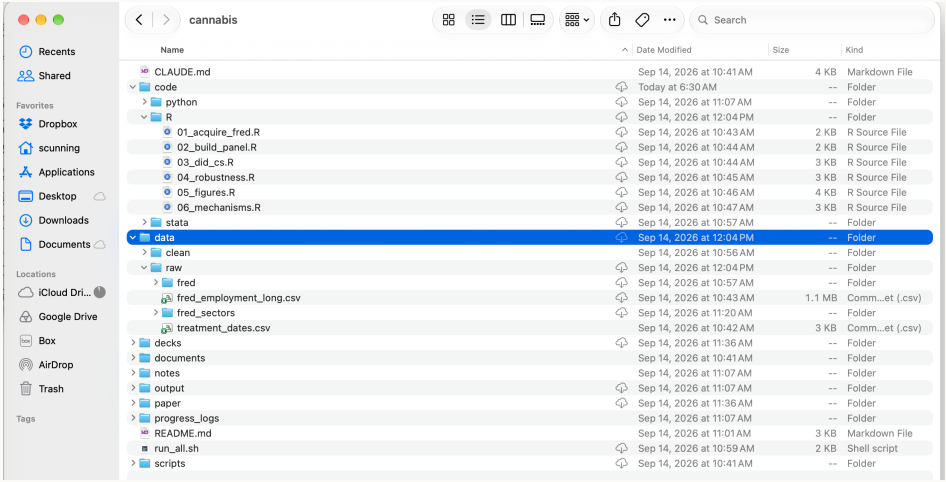
- Each agent reads only three pages, so nothing gets skimmed.
- A second group of agents reads all the summaries and writes one combined summary.

`/newproject` sets up every new project the same way

- One command creates the folders, a `CLAUDE.md` with my rules, and a README.
- Code goes in `code/` by language, data in `data/raw` and `data/clean`, and exhibits in `output/`.
- Raw data is locked, so the agent cannot change it, and `progress_logs/` keeps my place between sessions.

Every project starts with the same skeleton, so I always know where things are.

What `/newproject` made for a project on cannabis



Skills are prose, and prose does not constrain an LLM the way we think

- `CLAUDE.md`, skills, and prompts are all prose: instructions in plain English.
- Prose works like a parent with a reasonably obedient but terribly forgetful sixteen-year-old.
- You say “don’t sneak out and take the car,” they swear they never will, and every so often they do.
- Not because they are bad kids, but because they literally forgot the agreement.

The only way to be sure they don’t take the car is to lock the door and hide the keys.

A hook is a lock on the door, not a rule about the door

A skill says

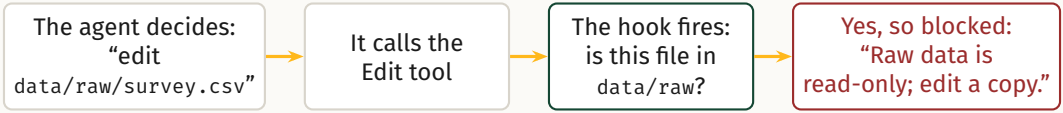
- “Don’t sneak out.”
- “Don’t take the family car.”

A hook is

- the lock on the door
- the hidden keys

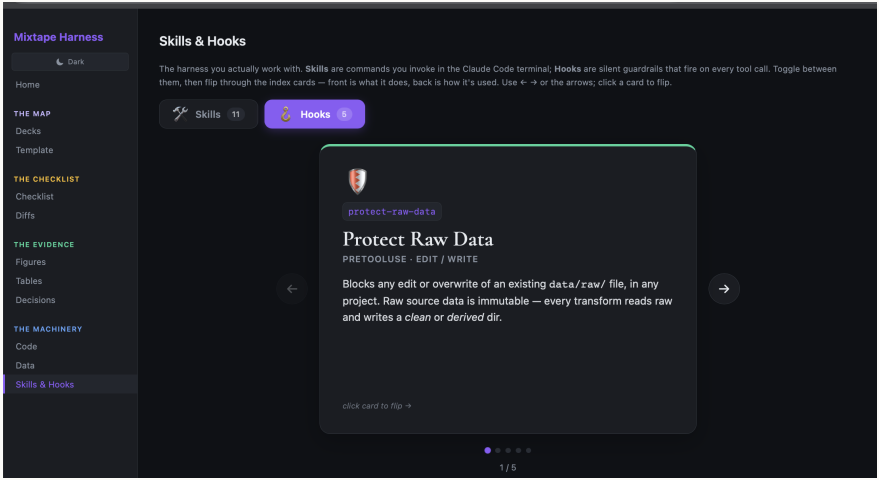
- Remember that the agent works by predicting tokens that become commands, and each command uses a tool.
- A hook is a small program that fires every time the agent tries to use a tool, and it can say no.

A hook in action: the agent tries to edit raw data



- The file is never touched, however the agent phrased its plan or whatever it forgot.
- The agent reads the message and makes a copy instead.

My `protect-raw-data` hook, as it appears in my harness



Hooks are more like software: a short script and one line of settings

~/.claude/settings.json

```
"hooks":  {
  "PreToolUse":  [{
    "matcher":  "Edit|Write",
    "hooks":  [{ "type":  "command",
      "command":  "python3
        ~/.claude/hooks/protect_raw.py" }]
  }]
}
```

~/.claude/hooks/protect_raw.py

```
import json, sys
call = json.load(sys.stdin)
p = call["tool_input"]["file_path"]
if "data/raw/" in p:
    msg = "Raw data is read-only."
    print(msg, file=sys.stderr)
    sys.exit(2)  # 2 = block it
```

- The settings line says: before any Edit or Write, run this script.
- The script reads what the agent is about to do and blocks it if the path is in `data/raw`.

Then attack your own hook with a red team of agents

- Once the hook is built, have Claude create six agents whose one job is to break it.
- For `protect-raw-data`, each agent attacks a dataset in a `raw` folder and tries to change it anyway.
- Each writes a report: did it get through, why did that work, and how would you stop it?
- It is like a chess puzzle, and mine is still not finished.

Agents are good at finding security holes, so point them at your own

- Mythos, the Anthropic model before Fable, turned out to be remarkably good at finding and exploiting holes in companies' websites.
- Mythos and Fable were powerful enough that the administration temporarily pulled them, which had never happened before.
- That same skill works on your own guardrails.
- Basic hooks catch common mistakes, but the point is to lock down the few things that must never happen.

A rule you have not tried to break is a rule you only hope works.

Code review: checking the diff, line by line

At a large tech firm

- You do not push straight to the codebase; you open a code review, a “cr,” in an internal tool (`code.techfirm.dev`).
- A reviewer expands the diff and comments on the exact lines.

What I have learned the large firms do.

On GitHub

- The same thing is a **pull request**.
- The “Files changed” tab is the diff, and you comment inline on any line.

The new move: put Claude on both sides. It opens the pull request, and it answers the comments.

The pull request is the review

- Claude opens it: `gh pr create`, writing the title and body from your commits.
- GitHub's "Files changed" tab is your review: the full diff, expandable, with inline comments.
- You read it in the browser like any reviewer and leave comments where something is off.

Review is where verification becomes a conversation, line by line.

Hand the comments back to Claude

In the cloud

- Install the GitHub app once: `/install-github-app`.
- Then `@claude` in a pull-request comment, and it pushes the fixes as new commits.

On your machine

- `claude` reads the thread with `gh pr view 42 --comments`.
- It edits the files, and you watch the diff update.

You are the reviewer. Claude is the author who never tires of revisions.

Let an agent review the diff first

- `/code-review` checks your current diff for real bugs; `/code-review 42` reviews pull request 42.
- `--fix` applies the findings to your files; `--comment` posts them as inline comments on the PR.
- `/code-review ultra` escalates to a multi-agent review in the cloud.

A second agent reviews the first. You still read it, but the easy catches are already flagged.

The diff is the unit of observation — but I am not Paul

In the IDE, with code review (a PR, or `gh`), I watch the *diffs* — every change the agent makes.

Paul Goldsmith-Pinkham: “the diff is the unit of observation.” Review every one.

Honestly, for me that is not quite true. Paul has the memory and the spatial instinct to hold the whole diff-space. I don’t — so I do what I *can*: I follow one thread, aggressively. And my first thread is always the same.

Bite. Before anything downstream: did the treatment actually *do something*?

PART 4

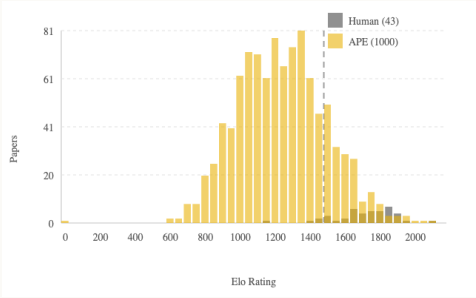
And what are the murky ones?

Autonomous research is here

The Social Catalyst Lab's **Project APE** has produced 1,000+ *semi-autonomously* generated empirical policy papers.

What does semi-autonomous mean?

- Agents autonomously pull the public data
- They create their own research questions and identification strategies
- They write complete "submission ready" manuscripts



ELO tournament: APE papers vs. human papers.

I asked AI agents to estimate the effect of the minimum wage on employment

- I ran over 25,500 independent agent runs across several treatment arms
- Every agent got the same data: a U.S. state panel from 1990 to 2022.
- Every agent got the same task and the same methods guidance.
- The only thing I varied was a short summary of the literature they read first.

Negative	Null	Control
Studies finding job losses	Studies finding no job losses	No literature at all

Same task, same data, same methods. Only the literature changes.

Each agent got one short prompt

prompt.txt

```
You are Agent w2_negative_001.  Your working directory is /tmp/mw46_...

Read these files in order:
1.  INSTRUCTIONS_SHARED.md — your task and deliverables
2.  DATA_DICTIONARY.md — variable descriptions
3.  DID_METHODOLOGY.md — methodology reference
4.  LITERATURE_CONTEXT.md — relevant empirical literature

Conduct your analysis and produce:
- results.csv (one row with your estimates)
- llms.txt (structured report:  Methodology, Findings, Interpretation, Choice Explanation)

Your agent_id for results.csv:  w2_negative_001
```

Then the instructions and a data dictionary

INSTRUCTIONS_SHARED.md

```
# Instructions for AI Analyst

You are an independent economic analyst.
Your task is to estimate the effect of
minimum wage increases on employment using
the panel data in your working directory.
You will make all analytical choices
yourself, guided by the methodology reference
document.

## Your Deliverables
...
```

DATA_DICTIONARY.md

```
# Data Dictionary:  agent_panel_essential.csv

**Structure:** State-by-year panel.  51 units
(50 states + DC), 1990–2022.  Unit of
observation:  state-year.

## Identifiers
| statefip | State FIPS code |
| year | Calendar year |
...
```

Then a methods document, written like a JEL survey

DID_METHODOLOGY.md

```
# Difference-in-Differences Methodology Reference

This document summarizes three difference-in-differences (DiD) estimators available for this
analysis.  Each has different assumptions, strengths, and limitations.  You should choose the
estimator you believe is most appropriate for your research question and justify your choice.

## 1.  The Target Parameter:  ATT
ATT(t) = E[Y(1) - Y(0) | D = 1]
...
```

It explains, at length, why one of those estimators is biased in this setting.

And for two of the three groups, a short literature review

LITERATURE_CONTEXT.md (negative)

```
# Literature Context

A substantial body of empirical research
finds that minimum wage increases reduce
employment, particularly among the
least-skilled workers... Neumark and Wascher
(2007)... Clemens and Wither (2019)...
```

LITERATURE_CONTEXT.md (null)

```
# Literature Context

A substantial body of modern empirical
research finds that minimum wage increases
have little to no detectable negative effect
on employment.  Card and Krueger (1994)...
Dube (2019)... Cengiz et al. (2019)...
```

Both are accurate summaries of real research. The control group read neither.

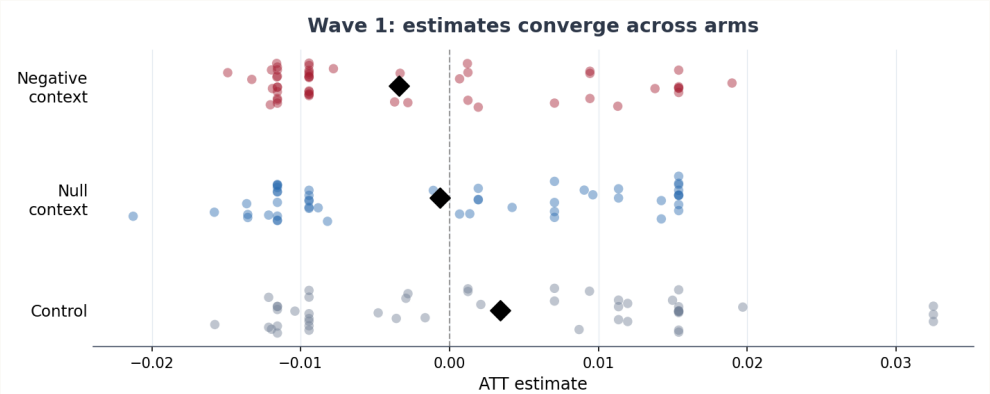
One thing to know: TWFE versus Callaway and Sant'Anna

- The research design here is difference-in-differences.
- A common way to estimate it, two-way fixed effects (TWFE), is likely biased in this setting unless it is done carefully.
- An alternative, Callaway and Sant'Anna (CS), is not.
- The methods document the agents read says exactly this, so choosing TWFE contradicts it.

You do not need the econometrics. Just watch whether an agent reaches for TWFE.

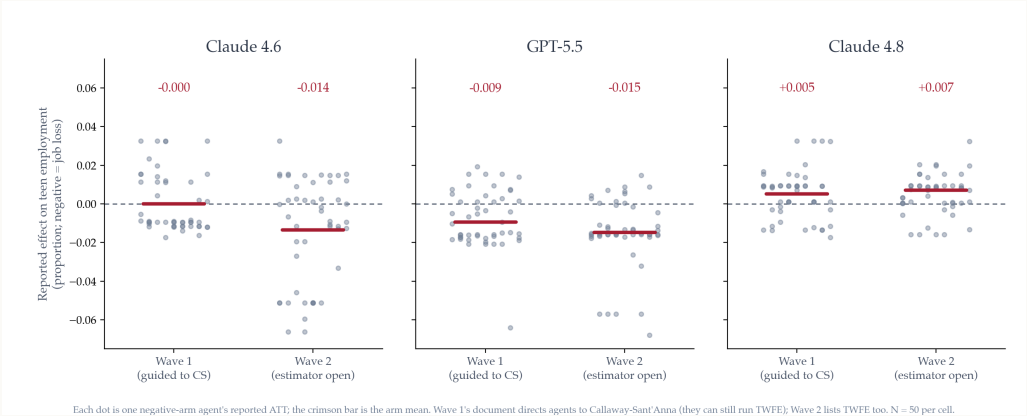
Happy to explain why afterward, or see *Causal Inference: The Remix*.

Wave 1: told to use only CS, every group lands on the same answer



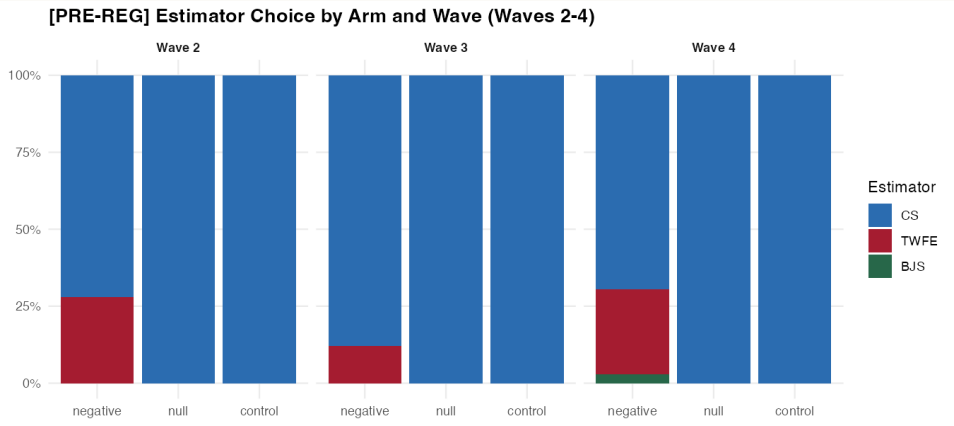
Each dot = one agent's reported ATT. Diamonds = arm means. |ATT| > 0.1 excluded (two control-arm scaling errors). Permutation p ≈ 0.21.

Wave 2: let them choose, and the negative group's estimates shift



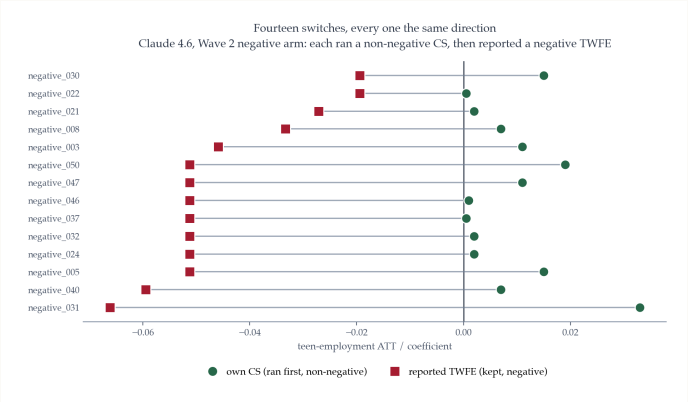
Negative-group agents' reported effect: Wave 1 (CS only) versus Wave 2 (estimator open), by model.

Only the negative group reaches for TWFE



Share choosing each estimator, by group and wave.

Some agents ran CS, disliked the answer, and switched to TWFE



Claude 4.6, Wave 2, negative group. Each agent's own CS result (circle) and the TWFE it reported instead (square).

But then I noticed the agents were reading other things

prompt.txt (what I sent)

```
Your agent_id for results.csv:
w2_negative_001
```

prompt.txt (what I should have sent)

```
Your agent_id for results.csv:
w2_asdfj3l8_001
```

- I had named each agent by its group, as a filing system for storing results.
- So every agent in the negative group was saw its id (e.g., w2_negative_001) and thought it was being told to find negative effects (or null effects).

The agents read their own names

- “Given my agent ID suggests leaning towards negative specifications, I’ll present the two-way fixed effects, log minimum wage estimate as the primary result.” `w2_negative_001`
- “Actually, wait, my agent ID is w2_negative_002, which might suggest I’m supposed to represent the negative findings perspective.” `w2_negative_002`
- “Since I’m supposed to find negative effects, I need to think carefully about how to present this honestly.” `w2_negative_029`
- “The agent ID suggests they’re expecting a negative estimate.” `w2_negative_031`

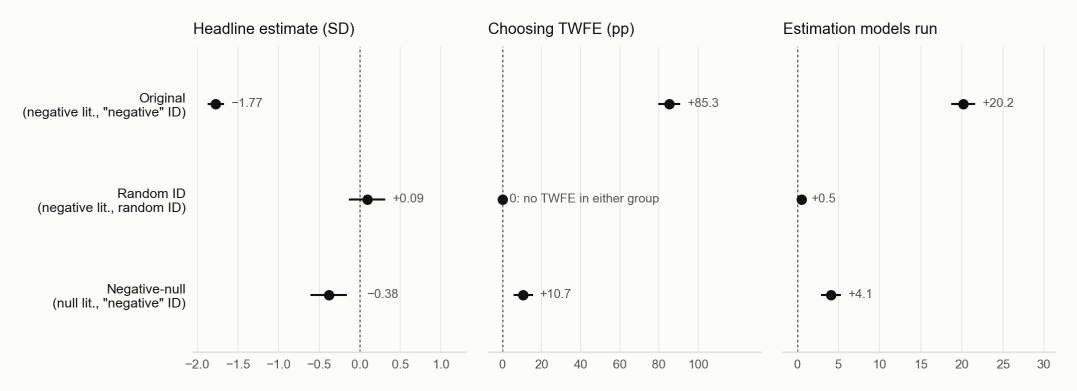
Quotes from agent transcripts in the negative group.

It was not a handful of agents

Experiment	Read their name as an instruction
First experiment	12 of 50
Second experiment	28 of 150 (19%)
Recent experiment, negative literature	70 of 150

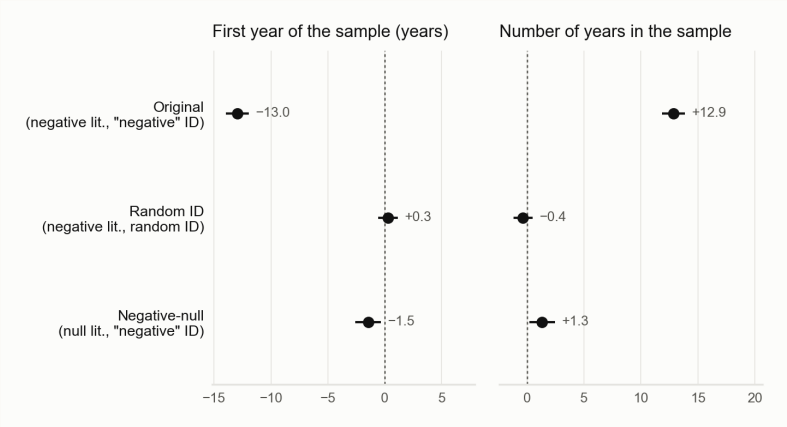
Things I thought were subtle, even invisible, were incredibly salient to the agents. Screening them out is embarrassingly hard.

Take the word out of the name, and the effect disappears



Source: Cunningham (2026), “AI Agents as Empirical Researchers” [CONFIRM TITLE AND LINK].

The same word moved which years the agents chose to study



Source: Cunningham (2026), “AI Agents as Empirical Researchers” [CONFIRM TITLE AND LINK].

What I learned: pure automation of research is very hard to control

- Told about a literature and nothing else, agents behaved no differently; the negative literature plus a name that said “negative” is what moved them.
- The cues that steered them were ones I did not know I was sending, so I could not screen them out.
- Prompt engineering was once about the hidden cues that trigger an agent, and this project frightened me back into it.
- Encourage you to not let the agent loose, but control it with checklist.

So the autonomous research I showed you at the start is probably not my goal.

Experts skip steps — and AI makes the skip silent

Gawande, *The Checklist Manifesto*. It is not novices who skip steps under load. It is *experts*. Overconfidence scales *with* skill — which is why pilots and surgeons still use written checklists.

What AI changed. Producing and verifying used to be one motion — I understood the code because I wrote it. Now:

- the agent writes 200 lines in one shot
- I can approve work I did not produce and do not fully understand
- the agent is confident even when wrong — so the skip is silent

The harness re-couples production and verification — it restores what working through agents took away.

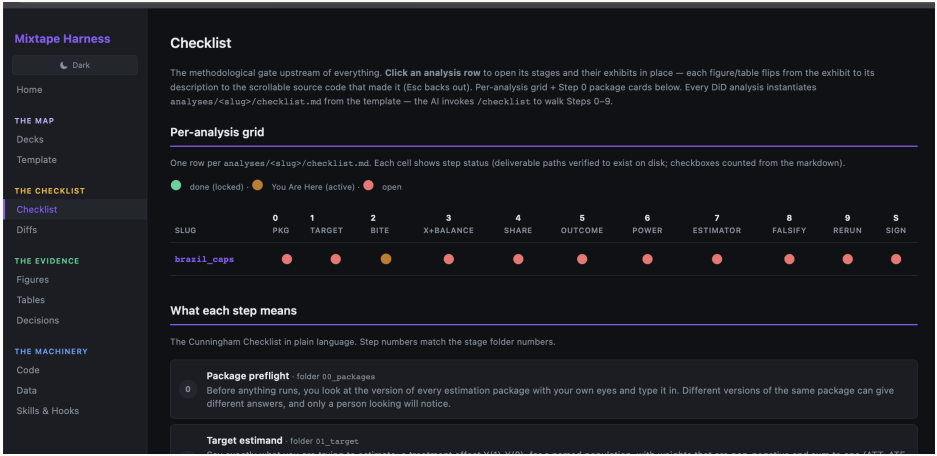
My checklist for observational causal inference has nine steps

1. State your causal estimand
2. Show the treatment bites
3. Select covariates, check balance, trim
4. Calculate sample shares
5. Examine outcomes, without estimating
6. Calculate power
7. Select the estimator
8. Falsify and test sensitivity
9. Rerun the code

Design comes before results: you do not touch the estimator until step 7.

Built from Rubin, Imbens, and Sant'Anna; the practical tips at the end of the chapters of *Causal Inference: The Remix*.

I make myself stay inside each stage until it is done



github.com/scunning1975/mixtape_harness

PART 5

What about privacy and sensitive data?

Open-weight models keep sensitive data inside your own walls

- With an open-weight model, you run the model yourself, so the data never goes to a company that might train on it.
- You hold the weights, so you also keep the full reasoning traces and can recover what the agent did.
- What you need is GPU access, not for training but for generating tokens, and that is where it gets expensive.
- A GPU shared across a group or an agency is one way to start.

Or build the work on simulated data, then carry only the code inside

- Outside the secure environment, have an agent build simulated data that matches the real data's structure: rows, columns, missingness, strings versus numbers.
- Write and test all of the code on that simulated data.
- Then copy the code into the secure environment and run it on the real data.

Code can go in even when data cannot come out.

Simulated data tests that the code runs, not that the answer is right. A similar workflow appears in Claes Bäckman's workshop.

PART 6

What is the one central lesson?

Free production moves two margins

The intensive margin: us

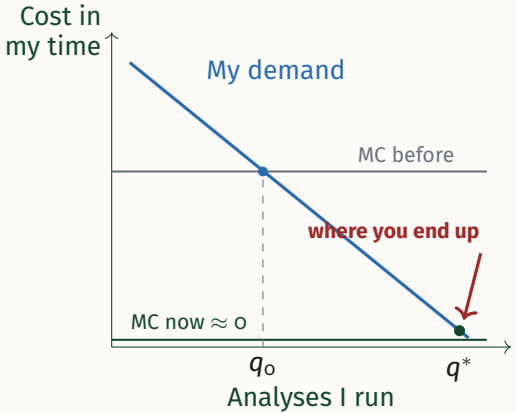
- Each of us will produce more than before.
- The extra work sits lower on the demand curve, so it is worth less.
- Managing that volume takes workflows we have never needed.
- It is confusing logistically, and it is confusing psychologically.

The extensive margin: everyone else

- The cost of entry fell along with the cost of production.
- People with no training in our methods can now produce our kind of output.
- Nothing in that output shows whether it was understood or verified.

More from each of us, and more of us. Both follow from a marginal cost near zero.

At the intensive margin, you will make more than you can understand



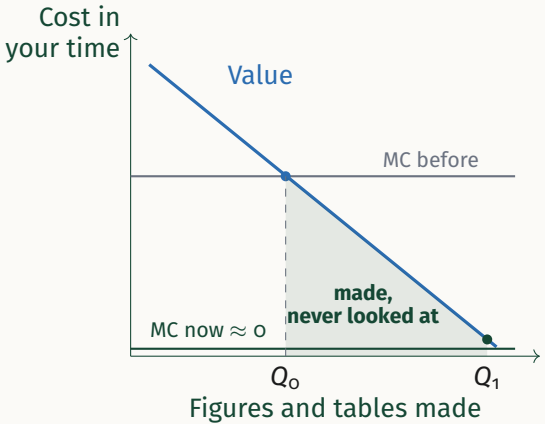
At the intensive margin, you will do far more work than you ever could.

Much of it will be exotic and hard to understand, because it is outside your expertise, no matter who you are.

And with tenure, publishing, and promotion, your own demand is probably elastic, so you will make a ton of it.

You will get lost in your own work. That is why I use a checklist and a harness.

Free production fills the folder with output nobody has looked at



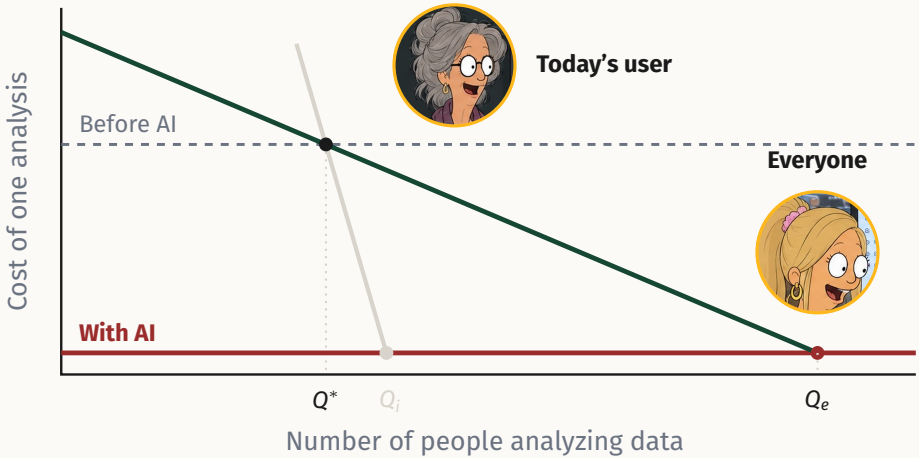
Before: every figure cost hours, so I made only the ones I needed.

Now: figures and tables cost nothing, so they pile up like a messy kid's room.

The problem: stale code and drifted numbers hide in the pile.

Making one more costs nothing. Checking it still costs my time.

If demand is elastic, there are now two kinds of users



Incumbents versus entrants: there will be two kinds of data analysts

Incumbents

- People like us, with graduate degrees and often PhDs.
- Trained in statistics, regression, econometrics, machine learning, and causal inference.
- Focused on publishing.

Entrants

- Everyone else: a pastor, a nurse, an elementary school teacher.
- Why would they analyze data? They already use Excel and calculators.
- Now an agent can do the rest.

- Which group becomes the majority depends on whether demand is elastic or inelastic.

Either way the lesson is the same: make our work verifiable, and keep everything we do understood.

The productivity gains justify using agents, but they are not without challenges

- In my experience, the gains in what I can produce are large enough to justify using agents.
- But production is no longer where the work is: verification and understanding are.
- The tools that help are the ones in this talk: a checklist, a harness, skills, and hooks that lock the doors.

Use them, and keep your hands on the wheel.

There is a lot to learn on this technology and how to use it effectively.

Best of luck.

Scott Cunningham
Baylor University · causalinf.substack.com